

# Manual técnico

## Desafio BQ

Arquitetura, mecânicas, execução local, exemplos e critérios verificáveis para implementar, revisar e manter a experiência.

BASE AUDITADA

31 AGO 2026

Aplicação web de página única, sem backend.

15 TESTES + BUILD

SYS.01

# Visão técnica

Desafio BQ é uma SPA para a Expô Bentinho 2026. A experiência combina um minigame em Canvas, controlado por clique, toque, teclado ou movimento facial, e um quiz de cinco perguntas com dicas conquistadas no jogo.

Todo o estado vive no navegador. Não há roteador, API própria, autenticação, banco de dados ou placar remoto. App .tsx funciona como máquina de estados e troca as telas por renderização condicional.

|                                          |                                              |                                    |
|------------------------------------------|----------------------------------------------|------------------------------------|
| <div>09</div> <div>estados de tela</div> | <div>70</div> <div>perguntas validadas</div> | <div>30s</div> <div>minigame</div> |
|------------------------------------------|----------------------------------------------|------------------------------------|

Responsabilidade central. App .tsx coordena progressão, pontuação, dicas, timers e ciclo do rastreador. Regras puras ficam nos motores de quiz, colisão, dificuldade e calibração.

## Linguagens e plataforma

TypeScript estrito modela contratos e lógica; TSX descreve as telas React; CSS nativo implementa identidade, layout e responsividade; Canvas 2D desenha o minigame; HTML5 fornece o shell e as APIs de mídia.

## SYS.02

## Stack e dependências

Versões resolvidas no lockfile e instaladas durante a auditoria. Como o `package.json` usa `latest` na maioria dos pacotes, `npm ci` é o comando reproduzível.

| Tecnologia             | Versao | Papel                                         |
|------------------------|--------|-----------------------------------------------|
| TypeScript             | 7.0.2  | Tipagem estrita e contratos dos motores.      |
| React / React DOM      | 19.2.8 | Componentes, estado e efeitos das telas.      |
| Vite                   | 8.2.2  | Servidor, transformação e bundle.             |
| MediaPipe Tasks Vision | 1.0.1  | Face Landmarker para a coordenada do nariz.   |
| Vitest                 | 4.1.11 | Testes unitários e integrados em JSDOM.       |
| Testing Library        | 16.3.3 | Interações e asserções orientadas ao usuário. |
| Canvas 2D / CSS        | Nativo | Jogo, layout, animação e responsividade.      |

Dependência de rede no modo câmera. O WASM é baixado de `cdn.jsdelivr.net` e o modelo de `storage.googleapis.com`. Quiz e modo clássico não precisam desses arquivos.

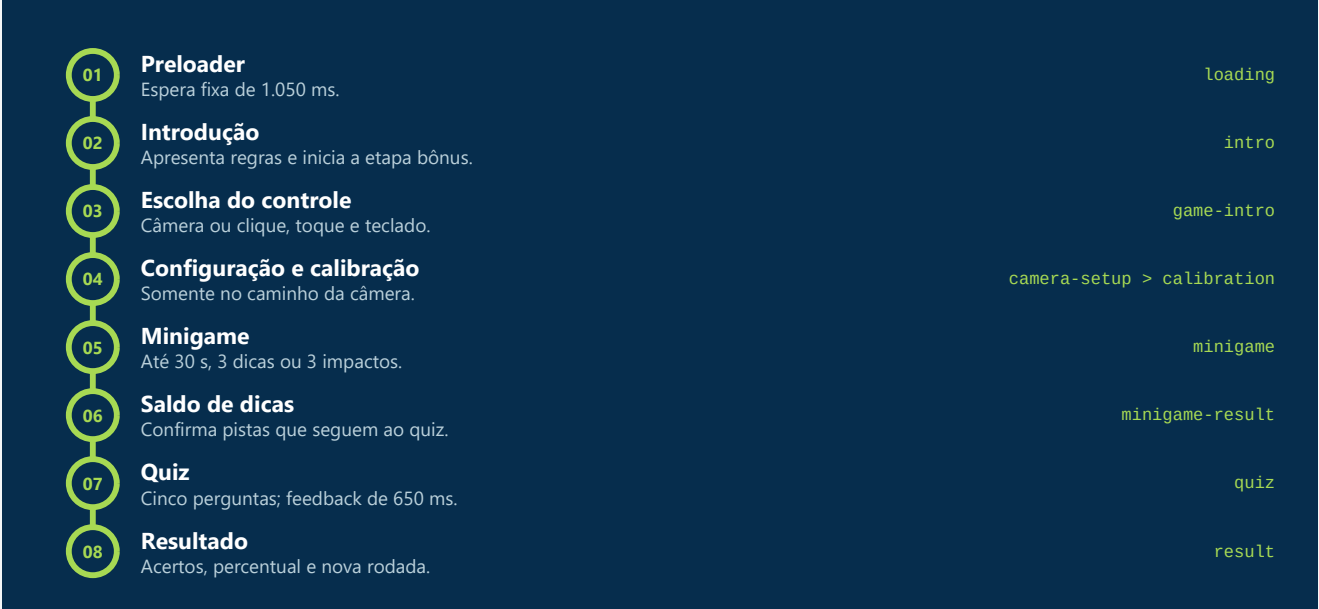
### Requisitos

- Node.js 20.19 ou superior dentro da faixa aceita pelo Vite 8; uma versão LTS compatível é preferível.
- Navegador moderno com ES2022, módulos, Canvas 2D e `requestAnimationFrame`.
- Para câmera: `getUserMedia`, localhost ou HTTPS, permissão do usuário e acesso às duas origens externas.

SYS.03

# Fluxo da experiência

A sequência é linear, com uma bifurcação de controle antes do minigame e fallbacks para o modo clássico. Cada troca de tela reposiciona o documento no topo.



Se um estado inválido impedir a renderização esperada, o fallback final de App volta à introdução com a mensagem para reiniciar a experiência.

## ENG.01

# Mecânicas - minigame e controles

## Loop do jogo

GameEngine executa com `requestAnimationFrame`. O delta é limitado a 50 ms para evitar saltos grandes. O canvas lógico mede 960 x 540; o CSS adapta a exibição.

- A dificuldade sobe aos 10 s e 20 s: velocidade aumenta, abertura e intervalo de spawn diminuem.
- Obstáculos, coletáveis e partículas que saem da tela são removidos.
- A rodada termina por tempo, ao coletar 3 dicas ou no 3º impacto.
- Cada impacto concede 1,05 s de invulnerabilidade e feedback visual.

## Controle clássico

Aplica gravidade de  $980 \text{ px/s}^2$  e impulso de -305 em `pointerdown` ou Espaço. A queda é limitada a 520 e as bordas amortecem a velocidade.

## Controle facial

Solicita câmera frontal 640 x 480, tenta GPU e recua para CPU. A leitura ocorre a cada 40 ms e usa o landmark 1, coordenada vertical do nariz.

- Calibração: mínimo de 12 leituras, cerca de 1,7 s e variação máxima de 0,018.
- Zona morta de 0,012; deslocamento normalizado em mais ou menos 0,16.
- Alvo vertical entre 58 e 482 px, com suavização exponencial.
- Sem rosto detectado, o último alvo válido é mantido.

## Colisão e coleta

Obstáculos usam interseção entre retângulos; dicas circulares usam o ponto do retângulo mais próximo do círculo. A hitbox é menor que o desenho do pássaro.

Inconsistência conhecida. A introdução afirma que colisões não encerram o jogo, mas o motor termina no terceiro impacto e o teste automatizado confirma a regra.

ENG.02

# Mecânicas - quiz, dicas e persistência

## Sorteio sem repetição

`createQuizRound` filtra IDs inválidos, remove perguntas usadas e garante cinco itens com ao menos uma pergunta de informática. Uma reserva dinâmica impede consumir perguntas técnicas necessárias para rodadas futuras. O ciclo reinicia depois de 70 perguntas.

## Banco validado

Ao importar `questions.ts`, a aplicação exige exatamente 70 itens, 25 de informática, 45 gerais, IDs únicos, quatro alternativas e índice de resposta válido.

## Dicas

Cada coletável vira uma dica. Ela só pode ser gasta antes da resposta e uma vez por pergunta. A pista é ocultada ao avançar, mas o saldo é preservado.

## Resposta e pontuação

O primeiro clique bloqueia alternativas, incrementa o placar quando correto e revela a opção válida. Em 650 ms avança ou abre o resultado. O percentual é `Math.round(score / 5 * 100)`.

## Persistência

A chave `desafio-bq:used-question-ids:v1` guarda apenas IDs usados. JSON inválido ou armazenamento bloqueado resulta em lista vazia; o jogo continua na sessão.

## ARC.01

## Estrutura de pastas

```
expo-bentinho2026/  
|- public/assets/           logos, brasão, assinatura e mascote  
|- src/  
|  |- App.tsx               estados e transições  
|  |- experience.types.ts   telas e modos de controle  
|  |- types.ts              contrato Question  
|  |- styles.css            tokens, layout e responsividade  
|  |- components/           Button, BrandHeader, HintCounter  
|  |- data/questions.ts     70 perguntas e validação  
|  |- utils/quizEngine.ts   sorteio, reserva e localStorage  
|  |- features/  
|    |- faceTracking/       câmera, calibração e suavização  
|    |- minigame/           motor, colisão, dificuldade, controles  
|  |- pages/                nove telas  
|  |- test/                 setup do jest-dom  
|- index.html               shell do Vite  
|- vite.config.ts           React e Vitest/JSDOM  
|- tsconfig*.json           TypeScript estrito e ES2022  
|- package.json             scripts e dependências  
|- package-lock.json        resolução reproduzível
```

Os testes ficam ao lado das unidades: `App.test.tsx`, `questions.test.ts`, `quizEngine.test.ts` e `gameEngine.test.ts`.

### Limites arquiteturais

- Não há roteador: atualizar a página reinicia a tela atual, embora preserve o ciclo de perguntas.
- Não há backend: placar e progresso não são fonte confiável para prêmios ou ranking.
- O estado de tela centralizado simplifica o fluxo atual, mas pode crescer demais se novas jornadas forem adicionadas.

## OPS.01

# Execução local

## 1. Verifique o ambiente

```
node --version  
npm --version
```

Use Node 20.19 ou superior dentro da faixa suportada pelo Vite 8.

## 2. Reproduza o lockfile

```
npm ci
```

Use `npm install` apenas quando a intenção for atualizar dependências e o lockfile.

## 3. Inicie

```
npm run dev
```

Abra a URL indicada pelo Vite, normalmente `http://localhost:5173`. Câmera exige localhost ou HTTPS.

## 4. Valide

```
npm test  
npm run build
```

Opcionalmente, `npm run dev -- --host 0.0.0.0` expõe o servidor à rede local. Não use o servidor de desenvolvimento em produção.



## DEV.01

## Exemplos de código

### Adicionar uma pergunta

```
{
  id: 'tech-26',           // Deve ser único.
  category: 'informatica', // 'informatica' ou 'geral'.
  prompt: 'Texto da pergunta',
  options: ['A', 'B', 'C', 'D'], // Exatamente quatro.
  correctAnswer: 0,         // Índice entre 0 e 3.
  hint: 'Uma pista curta.',
}
```

A validação exige 70 itens na proporção 25/45. Ao adicionar um item, remova ou reclassifique outro, ou atualize validação, motor e testes.

### Criar rodada com falha segura

```
try {
  const usedIds = loadUsedQuestionIds()
  const nextRound = createQuizRound(questionBank, usedIds)
  saveUsedQuestionIds(nextRound.usedIds)
  setRound(nextRound.questions)
} catch (error) {
  console.error(error) // Diagnóstico técnico.
  setError('Não foi possível preparar a rodada.')
}
```

Ao integrar uma API, transforme a resposta em `Question[]`, valide formato e contagens, trate timeout e status HTTP, e mantenha uma mensagem de recuperação.

## DEV.02

# Ciclo de vida e testes

## Liberar câmera e listeners

```
useEffect(() => () => {  
  window.clearTimeout(transitionTimer.current)  
  trackerRef.current?.stop()  
}, [])
```

FaceTracker.stop() para tracks, limpa timer e assinaturas, desconecta vídeos e fecha o landmarker.

## Sorteio determinístico

```
const round = createQuizRound(  
  questions,  
  previouslyUsedIds,  
  seededRandom(123456), // Mesma sequência em toda execução.  
)  
  
expect(round.questions).toHaveLength(5)  
expect(round.questions.some(q => q.category === 'informatica')).toBe(true)
```

A fonte aleatória injetável permite comprovar regras do sorteio sem testes instáveis.

## QA.01

## Qualidade e verificação

| Verificacao | Resultado    | Escopo                                   |
|-------------|--------------|------------------------------------------|
| Vitest      | 15 aprovados | 4 arquivos; banco, motores e interface.  |
| TypeScript  | Aprovado     | Compilação estrita via tsc -b.           |
| Vite build  | Aprovado     | 42 módulos transformados; bundle gerado. |

### Cobertura funcional existente

- Contagem, categorias, IDs e respostas do banco.
- 14 rodadas sem repetição, reserva técnica e reinício de ciclo.
- Duração, limite de dicas e encerramento no terceiro impacto.
- Calibração e limites do mapeamento facial.
- Preloader, modo clássico, fallback, consumo de dicas e fim do quiz.

### Smoke test manual recomendado

- Percorra o fluxo em 360 x 800, 844 x 390, 768 x 1024 e 1440 x 900.
- Teste Tab, Enter, Espaço, toque e clique; confirme foco e ausência de overflow horizontal.
- Negue e autorize a câmera; valide fallback e processamento facial.
- Teste zoom 200%, movimento reduzido, retrato, paisagem e safe areas.
- Corrompa a chave local; o quiz deve continuar. Bloqueie as CDNs; o clássico deve funcionar.

SEC.01

# Erros, privacidade e segurança

| Tema           | Tratamento atual e implicacao                                                                      |
|----------------|----------------------------------------------------------------------------------------------------|
| Câmera         | Quadros processados no dispositivo; sem armazenamento pelo código. Tracks encerrados ao sair.      |
| Fallback       | Mensagens para permissão, ausência, câmera ocupada, navegador e modelo; modo clássico disponível.  |
| Armazenamento  | Somente IDs de perguntas; leitura e escrita protegidas por try/catch.                              |
| Fornecimento   | WASM e modelo externos exigem CSP, monitoramento e avaliação de hospedagem local.                  |
| Confiança      | Placar no cliente é manipulável. Backend autoritativo é necessário para uso sensível.              |
| Acessibilidade | Botões nativos, aria-live, roles, foco e movimento reduzido; Canvas não tem equivalência completa. |

Recomendação de produção. Defina Content-Security-Policy para scripts, WASM, modelo e mídia; publique em HTTPS; mantenha o fallback clássico; monitore falhas de inicialização sem registrar imagens ou landmarks identificáveis.

## REF.01

## Premissas, limites e manutenção

### Premissas

- Navegador moderno e uma única face por sessão de controle.
- Iluminação, enquadramento, desempenho e permissão influenciam o modo câmera.
- A aplicação é uma experiência local, sem sincronização entre dispositivos.

### Limitacoes

- Limpar dados do site ou trocar de navegador reinicia o ciclo das perguntas.
- Não existem contas, ranking, analytics próprio, persistência ou validação remota.
- O preloader usa tempo fixo e não representa progresso real de carregamento.
- O texto sobre impactos diverge do comportamento do motor.

### Manutenção segura

- Prefira faixas explícitas em vez de latest para upgrades controlados.
- Ao mudar 70 itens ou a proporção 25/45, atualize validação, testes e documentação.
- Novos estados devem limpar timers, listeners, streams e motores ao sair.
- Toda nova mecânica precisa de teste puro no motor e teste de fluxo quando afetar a interface.

Critério de entrega. TypeScript compila, os 15 testes continuam verdes, novos ramos recebem cobertura proporcional e o fluxo crítico passa no smoke test de desktop, mobile e fallback sem câmera.

## Fim da referência

Documento derivado do repositório local auditado em 31/08/2026.